

## **INSTITUTO DE EDUCACIÓN SECUNDARIA JOSÉ PLANES**

Departamento de Informática y Comunicaciones

Técnico Superior en Desarrollo de Aplicaciones Web

C/ Maestro Pérez Abadía, 2 - 30100 Espinardo - Murcia

T. 968 834 605 - 30010577@murciaeduca.es - www.iesjoseplanes.es

# **Memoria del Proyecto Final**

Desarrollo de Aplicaciones Web

## **Plataforma Web de Gestión de Citas y Cerrajería de Automoción**

---

### **Autores:**

Hugo García Ramos

Francisco José Hidalgo Corbalán

### **Profesor/a-coordinador/a:**

Fernando Domingo Ruiz Meseguer

Murcia, Abril de 2026

## Licencia y Agradecimientos

---

Esta obra está bajo una licencia de Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional.

### Agradecimientos

**Al equipo docente:** En especial a Susana y a Fernando Domingo Ruiz Meseguer, por su guía, paciencia y dedicación durante todo el ciclo formativo, orientándonos en los estándares de desarrollo profesional.

**A nuestros compañeros:** A todos los compañeros de clase de DAW por explotar este proyecto al máximo, realizar pruebas y aportarnos críticas constructivas que nos han ayudado a mejorar la robustez técnica de la aplicación.

**A la familia:** Por el apoyo incondicional, la comprensión y los ánimos transmitidos durante las largas noches de entrega, maquetación y depuración de errores de código.

## Contenido

1. 

---

Resumen extendido
2. Palabras clave
3. Introducción
4. Estado del arte / trabajos relacionados
5. Análisis de objetivos y metodología
6. Diseño y arquitectura del sistema
7. Diseño y resolución del trabajo realizado
8. Presupuesto y planificación
9. Conclusiones y vías futuras
10. Bibliografía / Webgrafía

## 1. Resumen extendido

---

El presente proyecto final de Grado Superior consiste en el diseño, desarrollo e implementación de una plataforma web integral dirigida a una empresa especializada en el sector de la cerrajería de automoción, concretamente en el clonado, reparación y programación de llaves y mandos de vehículos. El objetivo principal de la aplicación es automatizar y centralizar la gestión de clientes mediante un sistema dinámico de reserva de citas, optimizando así el flujo de trabajo del taller y reduciendo la carga de gestión manual.

Para su construcción, se ha optado por una arquitectura desacoplada que separa el Frontend y el Backend. El cliente web ha sido desarrollado utilizando el framework Angular (TypeScript) junto con Tailwind CSS para un diseño adaptativo y moderno. El servidor se ha implementado mediante el framework Symfony (PHP), exponiendo una API REST robusta que interactúa con una base de datos ligera y eficiente basada en SQLite. El resultado es una aplicación ágil, segura y scalable que responde con éxito a las necesidades tanto de los usuarios finales como de los administradores del negocio.

## 2. Palabras clave

---

Angular, Symfony, API REST, SQLite, Tailwind CSS, Programación de llaves, Automatización de citas, Cerrajería de automoción, Single Page Application (SPA).

## 3. Introducción

---

En la actualidad, la transformación digital es un requisito indispensable para la supervivencia y optimización de cualquier modelo de negocio. Sin embargo, sectores altamente técnicos y especializados, como la cerrajería y electrónica del automóvil, suelen quedar rezagados en este aspecto, dependiendo en gran medida de canales de comunicación tradicionales como llamadas telefónicas o mensajería instantánea para la organización de sus servicios.

Este proyecto nace de la necesidad real de modernizar estos procesos a través de una solución web robusta y accesible de punta a punta. La aplicación permite a los propietarios de vehículos consultar los servicios disponibles (duplicado de llaves tradicionales, programación de mandos, llaves inteligentes, etc.) y agendar una cita en tiempo real rellenando un formulario específico con los datos de su vehículo.

Desde la perspectiva académica, el desarrollo de esta plataforma ha permitido consolidar y

unificar los conocimientos adquiridos durante el ciclo de Desarrollo de Aplicaciones Web (DAW), enfrentando el reto de conectar un ecosistema Frontend basado en componentes reactivos y señales de Angular con un backend empresarial estructurado en Symfony.

## 4. Estado del arte / trabajos relacionados

---

El sector de la cerrajería de automoción en el entorno web se caracteriza por la falta de herramientas interactivas. La inmensa mayoría de los competidores actuales disponen únicamente de "páginas escaparate" o sitios web estáticos (landing pages). Estas webs se limitan a mostrar información de contacto, un número de teléfono y una lista genérica de servicios, obligando al cliente a llamar o acudir físicamente para consultar disponibilidad y precios.

El valor añadido y factor diferencial de este proyecto radica en la ruptura de esta barrera analógica. Al integrar un módulo completo de reservas nativo dentro de la misma web, se proporciona una exclusividad y accesibilidad en el mercado local que mejora radicalmente la experiencia del usuario. El cliente puede autogestionar su cita en cualquier momento del día, introduciendo los detalles específicos de la marca, modelo y año de su coche, asegurando que el taller reciba de antemano toda la información técnica necesaria para realizar la programación o el clonado de la llave de forma eficiente.

## 5. Análisis de objetivos y metodología

---

### 5.1. Objetivos del proyecto

Tomando como referencia la propuesta inicial del proyecto, los objetivos de esta aplicación se definen mediante verbos en infinitivo, estructurándose de la siguiente manera:

- Desarrollar una aplicación web SPA (Single Page Application) modular y reactiva utilizando el framework Angular para el cliente y Symfony para el servidor.
- Implementar un sistema de control de accesos y seguridad mediante autenticación de usuarios para diferenciar los roles de cliente y administrador.
- Diseñar un catálogo público interactivo (escaparate) que exponga los servicios técnicos de clonado y programación de llaves de vehículos.
- Crear un módulo de gestión de citas dinámico que permita a los clientes registrar, consultar

y anular sus reservas desde su perfil privado.

- Construir un panel de administración centralizado que otorgue al personal del taller la capacidad de supervisar, modificar el estado y gestionar todas las citas del sistema.
- Garantizar la persistencia y la integridad de los datos mediante el diseño y la explotación de una base de datos relacional SQLite.

## 5.2. Metodología de desarrollo

Para la ejecución de este proyecto se ha optado por una metodología de desarrollo por fases (enfoque iterativo e incremental). Dado que el desarrollo ha sido realizado de forma individual y colaborativa por los autores, esta organización ha permitido mitigar riesgos, asegurar la estabilidad del núcleo de la aplicación antes de añadir nuevas funcionalidades y gestionar el tiempo de forma eficiente.

El ciclo de vida del proyecto se dividió en las siguientes fases cronológicas:

1. **Fase de Análisis y Diseño:** Definición de requisitos, modelado de la base de datos (tablas de usuarios, citas y servicios) y diseño de los primeros borradores de la interfaz de usuario.
2. **Fase de Desarrollo del Backend:** Configuración del entorno en Symfony, creación de las entidades y desarrollo de los endpoints de la API REST para las operaciones CRUD de las citas y la autenticación.
3. **Fase de Desarrollo del Frontend:** Inicialización del proyecto en Angular, configuración de Tailwind CSS, creación de servicios para conectar con el backend y desarrollo de los componentes de la interfaz.
4. **Fase de Integración y Pruebas:** Conexión de ambas arquitecturas, depuración de errores de tipos (TypeScript) y validación de los formularios de reserva y flujos de login.

## 5.3. Estructura y arquitectura de la aplicación

La plataforma web se encuentra dividida lógicamente en tres grandes zonas o módulos independientes, garantizando la separación de responsabilidades y la seguridad de los datos:

- **Zona Pública (Escaparate):** Es la puerta de entrada para cualquier visitante de la web. Funciona como un escaparate digital donde se detallan los servicios especializados del taller e invita al usuario a registrarse o iniciar sesión para acceder al sistema de reservas.
- **Zona de Cliente (Panel Privado):** Área restringida mediante autenticación. Una vez que el usuario accede a su cuenta, dispone de un espacio personal donde puede rellenar el formulario de reserva de citas (introduciendo los datos específicos de su vehículo) y visualizar el historial y el estado actual de sus solicitudes pendientes.
- **Zona de Administrador (Panel de Control):** Vista exclusiva para los gestores del taller.

Desde este panel, el administrador tiene una visión global de la agenda, pudiendo revisar los detalles técnicos de cada coche y actualizar el estado de las citas, centralizando la gestión del negocio de forma digital.

## 6. Diseño y arquitectura del sistema

### 6.1. Modelo y Diccionario de la Base de Datos

Para garantizar una correcta persistencia relacional con SQLite, se diseñó un modelo relacional estructurado compuesto por cuatro tablas esenciales independientes pero interconectadas:

Tabla 1: `usuario`

| Campo    | Tipo de Datos | Restricciones                | Descripción                              |
|----------|---------------|------------------------------|------------------------------------------|
| id       | INTEGER       | PRIMARY KEY<br>AUTOINCREMENT | Identificador único del usuario.         |
| email    | VARCHAR(180)  | UNIQUE, NOT NULL             | Correo electrónico para el login.        |
| password | VARCHAR(255)  | NOT NULL                     | Contraseña cifrada mediante Bcrypt.      |
| roles    | TEXT (JSON)   | NOT NULL                     | Roles asignados [ROLE_USER, ROLE_ADMIN]. |

Tabla 2: `servicio`

| Campo | Tipo de Datos | Restricciones | Descripción         |
|-------|---------------|---------------|---------------------|
| id    | INTEGER       | PRIMARY KEY   | Identificador único |

| Campo           | Tipo de Datos | Restricciones | Descripción                                     |
|-----------------|---------------|---------------|-------------------------------------------------|
|                 |               | AUTOINCREMENT | del servicio técnico.                           |
| nombre          | VARCHAR(100)  | NOT NULL      | Nombre comercial (Clonado, Duplicado, etc.).    |
| descripcion     | TEXT          | NULLABLE      | Explicación técnica detallada del trabajo.      |
| precio_estimado | NUMERIC(10,2) | NOT NULL      | Coste orientativo del servicio para el cliente. |

**Tabla 3: `cita`**

| Campo       | Tipo de Datos | Restricciones                             | Descripción                          |
|-------------|---------------|-------------------------------------------|--------------------------------------|
| id          | INTEGER       | PRIMARY KEY<br>AUTOINCREMENT              | Identificador único de la cita.      |
| usuario_id  | INTEGER       | FOREIGN KEY<br>REFERENCES<br>usuario(id)  | Propietario de la reserva efectuada. |
| servicio_id | INTEGER       | FOREIGN KEY<br>REFERENCES<br>servicio(id) | Servicio técnico solicitado.         |
| fecha_hora  | DATETIME      | NOT NULL                                  | Fecha y hora reservadas por el       |

| Campo           | Tipo de Datos | Restricciones                      | Descripción                                       |
|-----------------|---------------|------------------------------------|---------------------------------------------------|
|                 |               |                                    | cliente.                                          |
| marca_vehiculo  | VARCHAR(50)   | NOT NULL                           | Marca del vehículo (ej. SEAT, BMW).               |
| modelo_vehiculo | VARCHAR(50)   | NOT NULL                           | Modelo exacto del coche (ej. León Mk2).           |
| anio_vehiculo   | INTEGER       | NOT NULL                           | Año de fabricación para codificación del chip.    |
| estado          | VARCHAR(20)   | NOT NULL<br>DEFAULT<br>'pendiente' | Estado actual: [pendiente, aceptada, completada]. |

## 6.2. Documentación de Endpoints de la API REST Real

A continuación se detallan las operaciones, URIs y verbos HTTP que implementa la API de la plataforma web:

| Método | Endpoint           | Descripción / Operación                            | Código Respuesta Esperable    |
|--------|--------------------|----------------------------------------------------|-------------------------------|
| POST   | `/api/users`       | Crear/Registrar un nuevo usuario en la plataforma. | 201 Created / 400 Bad Request |
| POST   | `/api/login_check` | Autenticación de usuarios.                         | 200 OK / 401 Unauthorized     |

| Método | Endpoint              | Descripción / Operación                                                   | Código Respuesta Esperable    |
|--------|-----------------------|---------------------------------------------------------------------------|-------------------------------|
|        |                       | Comprueba las credenciales y genera el token JWT.                         |                               |
| GET    | `/api/servicios`      | Obtener la lista completa del catálogo de servicios técnicos disponibles. | 200 OK                        |
| GET    | `/api/servicios/{id}` | Obtener los detalles específicos de un servicio técnico concreto.         | 200 OK / 404 Not Found        |
| POST   | `/api/citas`          | Registrar o agendar una nueva cita en el sistema.                         | 201 Created / 400 Bad Request |
| GET    | `/api/citas`          | Listar las citas guardadas (filtradas por contexto de usuario/admin).     | 200 OK                        |
| GET    | `/api/citas/{id}`     | Consultar el detalle técnico y estado de una cita específica.             | 200 OK / 404 Not Found        |
| PUT    | `/api/citas/{id}`     | Modificar, actualizar parámetros o cambiar el estado                      | 200 OK / 400 Bad Request      |

| Método | Endpoint          | Descripción / Operación                                    | Código Respuesta Esperable     |
|--------|-------------------|------------------------------------------------------------|--------------------------------|
|        |                   | lógico de una cita.                                        |                                |
| DELETE | `/api/citas/{id}` | Eliminar o cancelar definitivamente una cita del registro. | 204 No Content / 404 Not Found |

## 7. Diseño y resolución del trabajo realizado

### 7.1. Justificación técnica de las herramientas utilizadas

El ecosistema tecnológico elegido responde a un criterio de eficiencia, compatibilidad y adaptabilidad a un entorno de desarrollo real e independiente:

- **Persistencia (SQLite):** A diferencia de proyectos corporativos que requieren servidores de bases de datos masivos y costosos de mantener, para este modelo de negocio se seleccionó SQLite desde la fase inicial de diseño. Su arquitectura basada en un único archivo integrado en el servidor reduce a cero la sobrecarga de conexiones de red, agiliza los tiempos de respuesta para un volumen de datos moderado y simplifica el despliegue del backend en Symfony sin sacrificar la integridad relacional de los datos.
- **Servidor de API (Symfony):** Se escogió este framework de PHP por su robustez arquitectónica, su potente motor de mapeo objeto-relacional (Doctrine ORM) y su facilidad para estructurar de forma limpia un patrón de API REST que maneje de forma nativa formatos JSON y serialización de objetos complejos.
- **Cliente Web (Angular y Tailwind CSS):** Angular aporta una arquitectura rígida y predecible basada en componentes, servicios e inyección de dependencias, idónea para construir la lógica de una SPA. La reactividad mediante Signals optimiza el rendimiento en la interfaz de usuario, mientras que Tailwind CSS permite maquetar el diseño visual de forma ágil mediante clases de utilidad directamente en el HTML.

### 7.2. El núcleo del problema: Integración e interoperabilidad entre arquitecturas

La principal dificultad técnica detectada y resuelta durante el ciclo de vida del proyecto residió en la comunicación bidireccional entre el cliente frontend (Angular) y el servidor backend (Symfony). Al tratarse de dos plataformas que se ejecutan en entornos completamente aislados, lograr una sincronización perfecta requirió resolver complejos retos de integración:

### **7.2.1. Gestión de formatos y desajustes de tipos en fechas**

Uno de los puntos críticos fue el intercambio de información temporal (fechas y horas de las citas). Mientras que la base de datos y la API de Symfony trabajan internamente con formatos estandarizados ISO 8601, las fuentes nativas HTML5 para la entrada de datos en los formularios del frontend exigen un formato estrictamente plano y recortado (yyyy-MM-ddThh:mm). Se implementó una capa intermedia de formateo dinámico en los componentes de Angular para parsear y reestructurar de manera segura las cadenas de texto temporales antes de cada envío o lectura de la API.

### **7.2.2. Rigidez de tipos bajo el Modo Estricto**

El compilador de TypeScript se configuró con las directivas de tipado estricto más exigentes. El uso de propiedades opcionales o dinámicas en los formularios (como las notas técnicas o de observaciones del coche) provocaba errores de compilación iniciales. La solución implementada consistió en diseñar interfaces extendidas con firmas de índice flexibles, habilitando la actualización dinámica de los campos en el estado de los componentes sin perder la seguridad de tipado que protege al proyecto.

### **7.2.3. Manejo de errores y respuestas de la API**

En el frontend se diseñó un manejador de errores centralizado robusto (`handleError`) que intercepta las respuestas HTTP incorrectas (`HttpErrorResponse`). Este módulo deserializa las violaciones del servidor, traduce los códigos de estado HTTP a mensajes comprensibles para el usuario final y hace un desplazamiento suave hacia la parte superior de la pantalla para notificar el fallo de manera amigable.

## **8. Presupuesto y planificación**

---

### **8.1. Estimación Temporal por Fases de Desarrollo**

Para la correcta organización del proyecto, se planificó un total de 80 horas de trabajo efectivo

distribuidas estratégicamente a lo largo de las distintas fases lógicas descritas en la metodología:

| Fase de Desarrollo Técnico     | Horas Dedicadas | Porcentaje de Esfuerzo | Tareas Principales Asociadas                                  |
|--------------------------------|-----------------|------------------------|---------------------------------------------------------------|
| Fase 1: Análisis y Diseño      | 15 horas        | 18.75%                 | Modelado de base de datos relacional y diseño de mockups UI.  |
| Fase 2: Implementación Backend | 25 horas        | 31.25%                 | Configuración de Symfony, Doctrine ORM y endpoints REST.      |
| Fase 3: Desarrollo Frontend    | 30 horas        | 37.50%                 | Maquetación Angular, Signals, consumo de HTTP y Tailwind CSS. |
| Fase 4: Integración y Pruebas  | 10 horas        | 12.50%                 | Resolución CORS, formateo estricto de tipos e ISO de fechas.  |

## 8.2. Desglose del Presupuesto Económico

Para realizar el cálculo del presupuesto de la aplicación web, se toma como métrica el esfuerzo temporal estimado en 80 horas de trabajo efectivo, repartidas a lo largo de un desarrollo de aproximadamente dos semanas. Al tratarse de un rol de desarrollo Full-Stack junior, se establece un coste por hora estándar de 25,00 € / hora.

| Perfil Profesional                            | Horas Estimadas | Coste por Hora | Total             |
|-----------------------------------------------|-----------------|----------------|-------------------|
| Desarrollador /<br>Analista Web<br>Full-Stack | 80 h            | 25,00 €        | 2.000,00 €        |
| <b>Subtotal Personal</b>                      |                 |                | <b>2.000,00 €</b> |

Adicionalmente, se contemplan costes fijos asociados a la amortización y el desgaste de infraestructura de hardware propio estimados en 50,00 €, mientras que las licencias de software utilizadas (Angular, Symfony, SQLite y Antigravity IDE) tienen un coste de 0,00 € al ser herramientas de código abierto o comunitarias.

**Total de ejecución material estimado: 2.050,00 €**

## 9. Conclusions y vías futuras

### 9.1. Conclusiones técnicas y personales

El desarrollo de esta plataforma web ha supuesto la culminación de los estudios en el ciclo de Desarrollo de Aplicaciones Web. El mayor logro técnico ha sido superar con éxito la barrera de comunicación e interoperabilidad entre un frontend basado en componentes reactivos modernos (Angular) y un entorno backend desacoplado estructurado (Symfony), asegurando que el flujo de peticiones e intercambio de datos JSON sea fluido y seguro.

A nivel metodológico, trabajar por fases de forma flexible ha permitido adaptarse a un ritmo de desarrollo discontinuo, aprendiendo a priorizar los módulos críticos. A nivel personal, el proyecto ha ayudado a desarrollar habilidades de resiliencia frente a errores de depuración complejos.

### 9.2. Vías futuras y mejoras pendientes

De cara al futuro y si se dispusiera de mayor tiempo de desarrollo, se contemplan las siguientes líneas de mejora:

- **Integración de un Asistente Virtual de Inteligencia Artificial (IA):** Implementar un bot

conversacional integrado en la zona pública para guiar al usuario, resolviendo dudas frecuentes sobre el clonado de llaves y ayudando de manera proactiva al cliente a elegir el tipo de servicio correcto antes de rellenar el formulario de cita.

- **Pasarela de pagos en línea:** Integrar una API de pagos como Stripe para abonar una señal o el importe total del servicio al confirmar la cita previa.
- **Sistema de notificaciones automatizadas:** Conectar el backend con servicios de mensajería para enviar recordatorios automáticos por SMS o correo electrónico a los clientes 24 horas antes de su cita agendada.

## 10. Bibliografía / Webgrafía

- ---

Angular Framework Documentation. (2026). *Angular Signals and Component Architecture Guide*. Recuperado de <https://angular.dev>
- SQLite Consortium. (2026). *SQLite Distinctive Features and Integration*. Recuperado de <https://www.sqlite.org>
- Symfony Framework Documentation. (2026). *Creating REST APIs with Symfony and Doctrine ORM*. Recuperado de <https://symfony.com>
- Tailwind Labs. (2026). *Tailwind CSS Utility-First Framework Documentation*. Recuperado de <https://tailwindcss.com>